Dokumentacja języka Robot2D zaimplementowanego w SLANG

Kacper Smykla

8 stycznia 2026

Spis treści

1 Opis problemu i celu **2**

2 Specyfikacja języka **3**
2.1 Model robota i pojęcie stanu . . . 3
2.2 Składnia konkretna . . . 3
2.3 Składnia abstrakcyjna (AST) i domeny . . . 4
2.4 Reprezentacja wyniku . . . 4

3 Definicja semantyki **5**
3.1 Domeny semantyczne i notacja . . . 5
3.2 Semantyka denotacyjna . . . 5
3.3 Semantyka naturalna (operacyjna, big-step) . . . 6
3.4 Związek obu semantyk i kryterium poprawności . . . 7

4 Projekt rozwiązania i algorytmy **8**
4.1 Ogólny przepływ danych . . . 8
4.2 Parsowanie i konstrukcji AST . . . 9
4.3 Złożoność semantyki denotacyjnej . . . 9
4.4 Złożoność semantyki naturalnej (big-step) . . . 9
4.5 Funkcje pomocnicze: ruch i obrót . . . 9
4.6 Automatyzacja budowania i uruchamiania . . . 10

5 Uruchamianie i nasadzenie **11**
5.1 Wymagania . . . 11
5.2 Struktura katalogów . . . 11
5.3 Budowanie projektu . . . 12
5.4 Uruchomienie interpretera . . . 12
5.5 Uruchamianie testów . . . 13

6 Testy i wyniki **14**

7 Wnioski **15**

Rozdział 1

Opis problemu i celu

Celem projektu jest zaprojektowanie i uruchomienie domenowego języka programowania (DSL) do sterowania prostym robotem poruszającym się po płaszczyźnie 2D. Język udostępnia minimalny zestaw instrukcji pozwalający opisywać ruch (krok do przodu w aktualnym kierunku, ruch o zadanej liczbie kroków), zmianę orientacji (obrót w lewo/prawo) oraz kompozycję programów (sekwencja instrukcji, nawiasowanie, instrukcja `skip` jako brak działania). Programy w tym DSL służą do wyznaczenia stanu końcowego robota (pozycja (x, y) oraz kierunek), startując z ustalonego stanu początkowego, zwykle po użyciu instrukcji `reset`. Program nie przechowuje żadnych zmiennych poza stanem robota, który można otrzymać jako wynik wykonania instrukcji.

Specyfikacja SLANG wymaga oprócz sprecyzowania składni języka (w postaci sekcji parser specyfikacji), także zdefiniowania semantyki poza samą składnią. Język jest opisany poprzez reguły składni i semantyk, a następnie uruchamiany w postaci interpretera. W trakcie pracy wykorzystywane są dwa style opisu zachowania programu: semantyka naturalna (operacyjna typu big-step), opisująca przejście od programu i stanu wejściowego do stanu wynikowego, oraz semantyka denotacyjna, traktująca program jako funkcję na stanach. Taki podwójny opis pozwala w testach porównywać wyniki obu podejść i weryfikować spójność implementacji.

Implementacja interpretera powstaje przy użyciu narzędzia SLANG (Semantics-Based Language Generator), które na podstawie specyfikacji języka (w specyficznej składni SLANG) generuje kod w języku Java. Wygenerowane struktury obejmują między innymi struktury reprezentujące drzewo składni abstrakcyjnej (AST), mechanizmy wykonywania semantyki (funkcje semantyczne i relacje przejścia), a także elementy związane z systemem typów/warunkami poprawności. Do parsowania składni konkretnej wykorzystywany jest generator parserów ANTLR4: SLANG generuje gramatykę `.g4`, a następnie ANTLR tworzy parser i lekser używane do wczytywania programów z plików tekstowych. Wynikiem tych narzędzi jest powstanie kompletnego wygenerowanego w Java interpretera.

Proces budowania i uruchamiania jest zautomatyzowany skryptami powłoki (Bash/Git Bash na Windows). Skrypty realizują kolejne kroki: uruchomienie SLANG w trybie generacji kodu, uruchomienie ANTLR w celu wygenerowania parsera, kompilację kodu Java (przez JDK) oraz pakowanie do pliku `.jar`. Oprócz tego zostały przygotowane dwa skrypty - pierwszy z nich służy do uruchamiania interpretera w trybie interaktywnym (zamiast wskazywania plików), gdzie przerwanie (Ctrl+Z lub Ctrl+D) kończy nasłuchiwanie tekstu i uruchamia interpreter z podanym kodem. Drugi skrypt przygotowany został do wykonywania testów poprawności specyfikacji/semantyk, gdzie każdy plik testowy zawiera program w DSL, a wynik działania interpretera jest porównywany z oczekiwanym stanem końcowym. Dzięki temu projekt jest możliwy do uruchomienia i weryfikacji bez środowiska IDE, wyłącznie z linii poleceń, która wspiera shell skrypty bash.

Podsumowując, projekt łączy formalny opis języka (w postaci specyfikacji SLANG) i jego semantyki z praktyczną implementacją interpretera. Wykorzystuje narzędzia ekosystemu Java, generator semantyczny SLANG (klas Java), generator parserów ANTLR4 oraz skrypty automatyzujące budowanie i testowanie. Efektem końcowym jest działający interpreter języka robotów oraz komplet materiałów umożliwiających jego ponowne zbudowanie i uruchomienie na docelowym systemie.

Rozdział 2

Specyfikacja języka

W niniejszym projekcie zdefiniowano domenowy język programowania (DSL) służący do sterowania robotem poruszającym się po dwuwymiarowej siatce. Język jest celowo minimalistyczny: zawiera jedynie konstrukcje niezbędne do opisu ruchu robota, zmiany kierunku oraz składania programów z prostych instrukcji.

2.1 Model robota i pojęcie stanu

Stan robota opisuje jego położenie oraz orientację. W projekcie przyjęto następujący model stanu:

- pozycja (x, y) na siatce, gdzie $x, y \in \mathbf{Z}$,
- kierunek dir należący do zbioru {N, E, S, W}, odpowiadający orientacji robota w przestrzeni (jego rotacji).

Z punktu widzenia wykonania programu semantyka języka sprowadza się do przekształcania stanu wejściowego w stan wyjściowy. W typowych scenariuszach testowych program rozpoczyna się od instrukcji `reset`, która ustanawia stan początkowy $(0, 0, N)$.

2.2 Składnia konkretna

Programy składają się z instrukcji rozdzielanych średnikiem `;`. Nawiasy `( )` służą wyłącznie do grupowania i wymuszania kolejności łączenia sekwencji. W języku występują następujące słowa kluczowe:

- `skip` instrukcja pusta (brak efektu),
- `reset` ustawienie stanu robota na $(0, 0, N)$,
- `forward` ruch o jeden krok do przodu w aktualnym kierunku,
- `forward n` ruch o n kroków do przodu, gdzie n jest liczbą naturalną,
- `turnleft` obrót robota o 90° w lewo,
- `turnright` obrót robota o 90° w prawo,
- `S1; S2` sekwencja dwóch instrukcji/programów.

Składnię można opisać w postaci uproszczonej gramatyki (BNF):

$$
\begin{array}{rcl}
S & ::= & \texttt{skip} \\
 & | & \texttt{reset} \\
 & | & \texttt{forward} \\
 & | & \texttt{forward}\ n \\
 & | & \texttt{turnleft} \\
 & | & \texttt{turnright} \\
 & | & (S) \\
 & | & S\ ;\ S
\end{array}
$$

W praktyce parser budowany przez ANTLR4 jest generowany z gramatyki `.g4`, którą SLANG wytwarza na podstawie sekcji `parser` w specyfikacji języka. W specyfikacji uwzględniono również kolejność dopasowań, aby uniknąć konfliktu pomiędzy `forward` oraz `forward n` (wariant z parametrem musi być rozpoznawany wcześniej).

2.3 Składnia abstrakcyjna (AST) i domeny

Składnia abstrakcyjna reprezentuje program w postaci drzewa (AST), niezależnie od szczegółów zapisu tekstowego. W SLANG AST jest modelowane jako suma konstruktorów w sekcji `domains`. Główne domeny używane w projekcie to:

- `Stmt` - instrukcje (programy) języka,
- `Program` - opakowanie na program główny: `Prog[Stmt]`.

Domena `Stmt` obejmuje następujące instrukcje:

- `Skip`
- `Reset`
- `Forward1`
- `ForwardN[n]` (gdzie n pochodzi z tokenu liczbowego / liczby naturalnej)
- `TurnLeft`
- `TurnRight`
- `Seq[s1,s2]` (sekwencja instrukcji)

Taki zestaw jest minimalny i wystarczający do reprezentacji wszystkich programów zapisanych w składni konkretnej. Nawiasy z poziomu składni konkretnej nie tworzą osobnych węzłów w AST - służą jedynie do tego, by parser zbudował właściwą strukturę `Seq[...]`.

Język zakłada, że parametr n w instrukcji `forward n` jest liczbą naturalną ($n \geq 0$). Warunek ten jest weryfikowany na etapie sprawdzania poprawności (typowania) w SLANG, co pozwala wykryć błędy jeszcze przed wykonaniem semantyki programu. W efekcie programy zawierające niepoprawne wartości parametru są odrzucane, a interpreter nie przechodzi do fazy obliczania stanu końcowego.

2.4 Reprezentacja wyniku

Wynikiem wykonania programu jest stan końcowy robota. W projekcie stan jest wypisywany w postaci tekstowej, typowo jako trójka `(x,y,dir)`. Ten format jest wykorzystywany zarówno w ręcznej weryfikacji działania interpretera, jak i w automatycznych testach, które porównują uzyskany stan końcowy z oczekiwaną wartością.

Rozdział 3

Definicja semantyki

Semantyka języka Robot2D definiuje znaczenie programów poprzez opis ich wpływu na stan robota. Stan jest trójką (x, y, dir), gdzie $x, y \in \mathbf{Z}$ oraz $dir \in \{N, E, S, W\}$. W projekcie zastosowano (zgodnie z wymaganiami) dwie semantyki: semantykę denotacyjną (jako funkcje na stanach) oraz semantykę naturalną/operacyjną typu big-step (jako relację przejścia). Mnogość semantyk w języku pozwala zweryfikować poprawność zachowania języka przez porównanie wyników obu definicji w testach.

3.1 Domeny semantyczne i notacja

Specyfikacja zawiera domenę `Stmt`, do której należa wszystkie wymagane operacje:

- `reset` – resetuje stan robota do stanu początkującego czyli (0,0,N)
- `forward n` – idzie do przodu o n jednostek
- `forward` – idzie do przodu o jedną jednostkę, specjalny przypadek forward n
- `turnright` – robot zmienia orientacje skręcając w prawo
- `turnleft` – robot zmienia orientacje skręcając w lewo
- `s1 ; s2` – sekwencja (zbiór) instrukcji, służąca do grupowania więcej niż jednej instrukcji

Oprócz tego jest jeszcze domena `Program`, który służy do opisywania całego programu za pomocą `Stmt`.

W semantyce wykorzystujemy funkcje pomocnicze zdefiniowane w sekcji `code` opisujące zachowanie robota:

- $turnL(dir)$ – obrót kierunku odpowiednio w lewo,
- $turnR(dir)$ – obrót kierunku odpowiednio w prawo,
- $forward(st, n)$ – wykonanie ruchu o $n \geq 0$ w aktualnym kierunku,

3.2 Semantyka denotacyjna

Semantyka denotacyjna interpretuje każdą instrukcję jako funkcję przekształcającą stan wejściowy w stan wyjściowy. Dla instrukcji (`Stmt`) definiujemy funkcję:

$$[\![S]\!] : State \to State$$

a dla programu (`Program`) analogicznie:

$$[\![P]\!] : State \to State.$$

Znaczenie podstawowych instrukcji jest następujące:

- `skip` nie zmienia stanu:
$$[\![skip]\!](st) = st.$$
- `reset` ustawia stan początkowy:
$$[\![reset]\!](st) = (0, 0, N).$$
- `forward` wykonuje krok do przodu:
$$[\![forward]\!](st) = step(st).$$
- `forward n` wykonuje n kroków:
$$[\![forward\ n]\!](st) = stepN(st, n).$$
- `turnleft` i `turnright` zmieniają jedynie orientację:
$$[\![turnleft]\!](x, y, dir) = (x, y, turnL(dir)),$$
$$[\![turnright]\!](x, y, dir) = (x, y, turnR(dir)).$$

Kluczową konstrukcją języka jest sekwencja. Dla programu postaci $S_1; S_2$ definiujemy złożenie funkcji:
$$[\![S_1; S_2]\!](st) = [\![S_2]\!]\big([\![S_1]\!](st)\big).$$
Widać tu naturalne dopasowanie do interpretacji imperatywnej: najpierw wykonuje się S_1, otrzymuje stan pośredni, a następnie wykonuje się S_2.

3.3 Semantyka naturalna (operacyjna, big-step)

Semantyka naturalna opisuje wykonanie programu jako relację pomiędzy konfiguracją wejściową (instrukcja i stan) a stanem wynikowym. W projekcie przyjmujemy relację:
$$\langle S, st\rangle \Rightarrow st'$$
gdzie S jest instrukcją, st stanem wejściowym, a st' stanem wyjściowym.

Reguły big-step dla konstrukcji podstawowych odpowiadają bezpośrednio intuicji wykonania:

- `skip`:
$$\frac{}{\langle skip, st\rangle \Rightarrow st}$$
- `reset`:
$$\frac{}{\langle reset, st\rangle \Rightarrow (0, 0, N)}$$
- `forward`:
$$\frac{}{\langle forward, st\rangle \Rightarrow step(st)}$$
- `forward n`:
$$\frac{}{\langle forward\ n, st\rangle \Rightarrow stepN(st, n)}$$
- `turnleft` oraz `turnright`:
$$\frac{}{\langle turnleft, (x, y, dir)\rangle \Rightarrow (x, y, turnL(dir))}$$
$$\frac{}{\langle turnright, (x, y, dir)\rangle \Rightarrow (x, y, turnR(dir))}$$

Najważniejsza reguła dotyczy sekwencji instrukcji:
$$\frac{\langle S_1, st\rangle \Rightarrow st_1 \qquad \langle S_2, st_1\rangle \Rightarrow st_2}{\langle S_1; S_2, st\rangle \Rightarrow st_2}$$
Reguła ta formalizuje standardową intuicję wykonywania programu sekwencyjnego z przekazywaniem stanu pomiędzy instrukcjami.

3.4 Związek obu semantyk i kryterium poprawności

Semantyka denotacyjna i naturalna opisują to samo zachowanie języka w dwóch różnych formalizmach. Kryterium poprawności implementacji w projekcie jest następujące: dla każdego programu S i stanu st obie definicje powinny prowadzić do tego samego stanu końcowego:

$$[\![S]\!](st) = st' \quad \Longleftrightarrow \quad \langle S, st\rangle \Rightarrow st'.$$

W praktyce jest to weryfikowane automatycznie w testach: interpreter uruchamia program, a następnie porównuje uzyskane stany końcowe w obu semantykach. Jeżeli stany końcowe się różnią, wyrzucany jest wyjątek. Dodatkowo przygotowano skrypt testowy, który porównuje otrzymany stan końcowy programu z oczekiwanym.

Rozdział 4

Projekt rozwiązania i algorytmy

Projekt został zaprojektowany jako pipeline operacji prowadzący od budowy interpretera języka DSL za pomocą SLANG, do obliczenia stanu końcowego robota, które odbywa się za pomocą wczytania programu w wygenerowanym interpreterze . Poprzez użycie narzędzia SLANG (składnia, AST, typowanie, semantyka) jest opisana cała w jednej specyfikacji, które można łatwo podglądąć, a kod wykonawczy jest generowany w Javie. Z perspektywy architektury, projekt dzieli się na część generacyjną (budowanie języka) oraz część wykonawczą (uruchamianie programów i testów). Format przyjmowanej specyfikacji języka w SLANG umożliwia na embedding kodu w Java, co pozwala na pewną kontrolę nad generowanym interpreterem oraz na dodawanie funkcji pomocniczych, których implementacja za pomocą samego formatu specyfikacji byłaby zbyt ciężka / uciążliwa.

4.1 Ogólny przepływ danych

Wykonanie programu w DSL przebiega według następujących etapów:

1. **Wczytanie tekstu programu** z pliku lub standardowego wejścia.
2. **Parsowanie** składni konkretnej do postaci drzewa składni (parser ANTLR4).
3. **Budowa AST** zgodnie z domeną `Stmt` oraz opakowanie w `Program`.
4. **Weryfikacja poprawności** (typowanie / warunki statyczne), np. sprawdzenie, że parametr n w `forward n` jest liczbą naturalną, sprawdzanie poprawności typów (za pomocą klauzul judgement)
5. **Ewaluacja semantyki** programu:
   - wariant denotacyjny: obliczenie $[[P]](st)$ jako funkcji na stanach,
   - wariant naturalny (big-step): obliczenie relacji $\langle P, st\rangle \Rightarrow st'$.
6. **Wypisanie wyniku** w postaci stanu końcowego (np. `(x,y,dir)`).

Dla testów automatycznych wykonanie jest powtarzane dla wielu programów wejściowych, a wynik jest porównywany z oczekiwanym stanem końcowym. Testy automatyczne są zaimplementowane za pomocą skryptu powłoki Bash, który uruchamia w pętli interpreter dla każdego z programu, znajdującego się w wybranym folderze (testcases) i porównuje otrzymany wynik z oczekiwaną wartością (znajdującą się w tym samym folderze o pliku z tą samą nazwą). Za każdym razem kiedy interpreter wykonuje program, robi to w obu semantykach, po czym porównuje czy wynik jest taki sam pomiędzy nimi. W konsekwencji każdy program, który nie kończy się wyjątkiem jest poprawny. Otrzymanie wyjątku oznacza albo zgłoszenie błędy w programie, gdy system typowania wykrywa błąd, albo semantyki zwracają różniące się wartości.

4.2 Parsowanie i konstrukcji AST

Parsowanie realizowane jest przez ANTLR4 na podstawie gramatyki wygenerowanej przez SLANG. Gramatyka odwzorowuje BNF języka, a w definicji parsera zastosowano dwie praktyczne zasady:

- **Rozróżnienie** `forward` **oraz** `forward n`: reguła z parametrem liczbowym ma pierwszeństwo, aby uniknąć sytuacji, w której token `forward` zostałby natychmiast rozpoznany jako instrukcja bez argumentu.
- **Sekwencja o najniższym priorytecie**: produkcja odpowiadająca `S1;S2` jest umieszczona na końcu, co sprawia, że nawiasowanie steruje strukturą AST, a nie przypadkowy porządek dopasowań.

Nawiasy w składni konkretnej nie są reprezentowane jako osobny konstruktor w AST. Ich funkcją jest wymuszenie odpowiedniej struktury drzewa, tj. kontrola zagnieżdżenia konstruktorów `Seq`.

4.3 Złożoność semantyki denotacyjnej

Semantyka denotacyjna jest realizowana jako funkcje rekurencyjne na strukturze AST. Dla konstruktorów podstawowych (np. `Reset`, `TurnLeft`) obliczenie wyniku jest stałoczasowe i polega na konstrukcji nowego stanu. Dla sekwencji:

$$[[Seq(s_1, s_2)]](st) = [[s_2]]([[s_1]](st))$$

algorytm działa w sposób rekurencyjny: najpierw oblicza stan pośredni dla s_1, a następnie wykorzystuje go jako wejście do obliczenia wyniku s_2. Wykonanie programu o długości k (liczbie elementarnych instrukcji) ma złożoność czasową rzędu $O(k)$, przy założeniu, że pojedyncze instrukcje mają koszt stały. Dla `forward n` koszt zależy od implementacji funkcji pomocniczej (w projekcie realizowane jest to jako obliczenie przesunięcia o n w jednym kroku, a nie n iteracji), więc nadal traktowane jest jako operacja stałoczasowa względem struktury AST.

4.4 Złożoność semantyki naturalnej (big-step)

Semantyka naturalna jest realizowana jako procedura wnioskowania big-step. Z punktu widzenia wykonania, jest to również rekurencyjne obliczenie po AST, ale opisane regułami relacji przejścia. Dla sekwencji:

$$\frac{\langle s_1, st\rangle \Rightarrow st_1 \qquad \langle s_2, st_1\rangle \Rightarrow st_2}{\langle Seq(s_1, s_2), st\rangle \Rightarrow st_2}$$

algorytm jest analogiczny do denotacyjnego: wyznacza st_1 na podstawie s_1, a następnie wyznacza st_2 na podstawie s_2 i stanu pośredniego. Złożoność czasowa jest również liniowa względem liczby instrukcji.

4.5 Funkcje pomocnicze: ruch i obrót

Aby semantyka była czytelna i modularna (oraz SLANG pozwolił na kompilacje), logika ruchu i obrotu została wyodrębniona do funkcji pomocniczych:

- **Obrót** implementowany jest jako mapowanie kierunków, np. $turnL(N) = W$, $turnR(N) = E$.
- **Ruch** implementowany jest jako obliczenie nowej pozycji zależnej od kierunku:
  - dla N: $(x, y) \mapsto (x, y + 1)$ (lub $(x, y + n)$ dla `forward n`),
  - dla S: $(x, y) \mapsto (x, y - 1)$,
  - dla E: $(x, y) \mapsto (x + 1, y)$,
  - dla W: $(x, y) \mapsto (x - 1, y)$.

Wydzielenie tych funkcji upraszcza reguły semantyczne i ogranicza ryzyko niespójności: jeśli zmienia się konwencja osi lub kierunków, modyfikacja zachodzi w jednym miejscu. Obie semantyki, korzystają z tych samych funkcji pomocniczych, więc jakakolwiek różnica w wynikach otrzymywanych pomiędzy semantykami mogą wynikać tylko ze specyfikacji samej semantyki, a nie implementacji logiki ruchu.

Wszystkie funkcje pomocnicze mają stałą złożoność czasową, gdyż każda z nich zwraca nowy wynik w zależności od otrzymanego wyniku na podstawie instrukcji warunkowej switch case, która jest efektywniejsza od łańcucha porównań `if/else-if`.

4.6 Automatyzacja budowania i uruchamiania

Część narzędziowa projektu obejmuje skrypty powłoki, które automatyzują:

- uruchomienie SLANG i wygenerowanie kodu Java na podstawie specyfikacji (skrypt `build.sh`),
- wygenerowanie parsera i leksera przez ANTLR4 (skrypt `build.sh`),
- kompilację kodu Java (JDK) (skrypt `build.sh`),
- uruchamianie interpretera (skrypt `run.sh`) oraz uruchamianie testów wsadowych (skrypt `test.sh`).

Rozdział 5

Uruchamianie i nasadzenie

Rozdział opisuje wymagania środowiskowe oraz sposób budowania i uruchamiania projektu z linii poleceń. Projekt został przygotowany tak, aby dało się go odtworzyć na czystej maszynie bez użycia IDE - cały proces sprowadza się do uruchomienia skryptów powłoki, które kolejno generują kod, budują parser, kompilują Javę i uruchamiają testy.

5.1 Wymagania

Do uruchomienia projektu wymagane są:

- **Java Development Kit (JDK) 21+** – wykorzystywany do uruchamiania SLANG, ANTLR oraz do kompilacji wygenerowanego kodu Java (`javac`) i pakowania archiwów (`jar`).
- **SLANG** – generator języków DSL oparty o specyfikacje składni i semantyki. Uruchamiany jako aplikacja Java, a dostarczany jest jako biblioteka `.jar` w katalogu `lib/` w repozytorium SLANG.
- **ANTLR4** – generator parsera/leksera; uruchamiany na gramatyce `.g4` wygenerowanej przez SLANG. Do uruchomienia wygenerowanego parsera wymagane są biblioteki runtime ANTLR, dostarczany jest jako biblioteka `.jar` w katalogu `lib/` w repozytorium SLANG.
- **Git Bash / Bash** – powłoka do uruchamiania skryptów `.sh`. Na Windows projekt zakłada korzystanie z Git Bash.

W repozytorium znajduje się katalog `lib/` zawierający pliki JAR potrzebne do uruchamiania SLANG i ANTLR (w tym biblioteki runtime). Skrypty wykorzystują `lib/*` jako element classpath. Do uruchamiania i budowania interpreterów języka DSL ze specyfikacji SLANG służą skrypty w folderze `bin/`

5.2 Struktura katalogów

W typowej konfiguracji projektu używane są następujące katalogi:

- `languages/` — specyfikacje SLANG (`robot2d.txt`) oraz pliki wejściowe,
- `languages/lang/robot2d/` — katalog wyjściowy kodu wygenerowanego przez SLANG i ANTLR (pliki `.java`, `.g4`),
- `build/robot2d/` — katalog budowania (klasy `.class`, archiwum `.jar`),
- `testcases/` — przypadki testowe (programy w DSL i oczekiwane wyniki).

Dokładne nazwy katalogów mogą być parametryzowane w skryptach (zmienne `BUILD_PATH`, `LANG_SPEC_NAME` itd.).

5.3 Budowanie projektu

Budowanie obejmuje cztery kroki:

1. **Generowanie kodu przez SLANG** z pliku specyfikacji.
2. **Generowanie parsera przez ANTLR4** na podstawie pliku `.g4`.
3. **Kompilacja wygenerowanego kodu Java** do katalogu klas.
4. **Pakowanie do pliku JAR** umożliwiającego uruchomienie interpretera.

Przykładowe polecenia (wykonywane w katalogu głównym projektu) są następujące:

- uruchomienie SLANG (generacja kodu):

  ```
  java -cp "lib/*" slang.Main -d languages/lang/robot2d languages/robot2d-v1.txt
  ```

- uruchomienie ANTLR4 (generacja parsera):

  ```
  java -cp "lib/antlr4.jar" org.antlr.v4.Tool languages/lang/robot2d/Robot2D.g4
  ```

- kompilacja kodu Java:

  ```
  javac -encoding UTF-8 -d build/robot2d/classes -cp "lib/*" @build/robot2d/sources.txt
  ```

- utworzenie archiwum JAR:

  ```
  jar cfe build/robot2d/Robot2D.jar Robot2DMain -C build/robot2d/classes .
  ```

W praktyce powyższe komendy są zebrane w skrypt budowania `build.sh`, który automatyzuje tworzenie katalogów, zbiera listę źródeł `.java` i wykonuje komendy w poprawnej kolejności.

5.4 Uruchomienie interpretera

Interpreter uruchamiany jest z wykorzystaniem klasy głównej (np. `Robot2DMain.java`) i z parametrem classpath zawierającego zbudowane archiwum JAR oraz bibliotekę `lib/`. Program w DSL może być podany jako plik lub przez standardowe wejście (dla skryptu `run.sh`).

Przykładowe uruchomienie z programu w pliku:

```
java -cp "build/robot2d/Robot2D.jar;lib/*" Robot2DMain < program.txt
```

Przykładowe uruchomienie interaktywne (wprowadzanie programu do EOF):

```
java -cp "build/robot2d/Robot2D.jar;lib/*" Robot2DMain < input_from_stdin.txt
```

Wynikiem działania jest wypisanie stanu końcowego robota w formacie tekstowym, np. `(x,y,dir)`. Ten format jest wykorzystywany w testach automatycznych.

5.5 Uruchamianie testów

Testy można uruchamiać w trybie wsadowym poprzez skrypt `tests.sh`, który:

1. iteruje po plikach `.txt` w katalogu `testcases/`,
2. dla każdego pliku uruchamia interpreter i przechwytuje wynik,
3. normalizuje i porównuje uzyskany stan z oczekiwanym wynikiem z pliku `.expected`,
4. raportuje statystyki `PASS/FAIL` - ile testów skończyło się sukcesem lub porażką.

Przykładowe uruchomienie testów:

```
./bin/tests.sh
```

W przypadku systemu Windows, projekt zakłada uruchamianie skryptów w Git Bash. W skryptach wprost wskazywana jest ścieżka do JDK (zmienna `JDK_PATH`), co pozwala uniknąć zależności od konfiguracji zmiennych środowiskowych systemu.

Rozdział 6

Testy i wyniki

Weryfikacja poprawności języka Robot2D została przeprowadzona w sposób praktyczny, poprzez uruchamianie interpretera na zestawie programów testowych. Testy miały trzy cele:

- potwierdzić poprawność składni i parsowania,
- potwierdzić poprawność obliczania stanu końcowego robota,
- potwierdzić spójność obu zdefiniowanych semantyk.

Na poziomie uruchomień ręcznych (wywołanie pętli głównej interpretera w Javie) zaobserwowano, że żaden z programów testowych nie kończy się porażką (poza jednym co się celowo kończy wyjątkiem). Oznacza to, że programy zawierają poprawną składnię zgodną z parserem oraz przechodzą etapy walidacji przed wykonaniem (np. ograniczenia dla parametrów liczbowych, poprawność typów). Dodatkowo dla większości testów interpreter wyznacza stan końcowy bez błędów wykonania,a tam gdzie błąd jest uzyskiwany (dla operacji forward 0), jest otrzymywany oczekiwany wyjątek, gdyż jest to nie zgodne ze specyfikacją, czego zabezpieczenie w kodzie jest sprawdzone praktycznie za pomocą testu.

Istotnym elementem testów była weryfikacja zgodności semantyk: dla każdego programu obliczany wynik semantyki denotacyjnej oraz semantyki naturalnej (big-step) był identyczny. Jest to praktyczne potwierdzenie, że dwie niezależne definicje znaczenia programu opisują to samo zachowanie języka (dla rozważanego podzbioru programów testowych).

Ostatecznie, automatyczny skrypt testujący (`test.sh`) uruchamia interpreter dla każdego pliku `.txt` w katalogu testcases oraz porównuje wypisany stan końcowy z wartością oczekiwaną. Wszystkie przypadki testowe zakończyły się wynikiem `PASS`. W połączeniu z brakiem wyjątków oraz zgodnością semantyk daje to spójny obraz poprawnego działania języka w ramach przyjętego zakresu funkcjonalnego.

Rozdział 7

Wnioski

Projekt zakończył się zbudowaniem działającego interpretera dla DSL Robot2D opartego o formalną specyfikację w SLANG oraz parser generowany przez ANTLR4. Z punktu widzenia celów semantycznych, kluczowym rezultatem jest to, że semantyka denotacyjna i semantyka naturalna (big-step) prowadzą do tych samych stanów końcowych dla wszystkich przygotowanych programów testowych, a dodatkowo wyniki są zgodne z oczekiwanymi wartościami referencyjnymi.

W praktyce praca z narzędziem SLANG okazała się wymagająca głównie ze względu na niewielką liczbę dostępnych, kompletnych przykładów specyfikacji, które można traktować jako wzorzec. Przy projektowaniu własnego języka największą trudność stanowiła iteracyjna diagnostyka błędów specyfikacji oraz dopasowanie składni sekcji (np. parsera i funkcji) do oczekiwań narzędzia. Wpływa to na czas potrzebny do rozwoju języka i zwiększa znaczenie testów oraz automatyzacji budowania.

Wykonując to zadanie, w poprzedniej (nieopublikowanej) wersji projektu miałem czystszy kod, jednak kosztem niespełnienia wszystkich wymagań. Osiągałem to poprzez pominięcie w sekcjach `domains`, `printer` oraz `parser` definicji `State` i `Dir`, a następnie przeniesienie ich do sekcji `code`. Zamiast polegać na definicjach generowanych przez SLANG, zdefiniowałem ręcznie `State` jako publiczny rekord (co wymaga nowszej wersji Javy) oraz `Dir` jako `enum`. W tej wersji nie miałem problemów z dostępem do pól rekordu, które pojawiają się w wersji obecnej: nie musiałem odwoływać się do pól przez rzutowanie i dopasowanie wzorca (`instanceof` pattern matching) ani korzystać z mało czytelnych nazw akcesorów zależnych od kolejności, np. `_1()`, `_2()`.

Dodatkowo rekord wygenerowany przez SLANG w praktyce traktował pola `x` i `y` jako wartości tekstowe (`String`), mimo że semantycznie są to liczby. Stało to w kontraście do mojego ręcznego rekordu, w którym typy `x` i `y` były jawnie liczbowe (np. `int`), co pozwalało uniknąć zbędnych konwersji typów. Pomimo że w obu wersjach wykorzystywałem rekordy (raz generowane przez SLANG, raz zdefiniowane przeze mnie w sekcji `code`), z mojej perspektywy doświadczenie developerskie (DX) było wyraźnie lepsze przy własnej definicji tych typów. Sugeruje to, że SLANG mógłby inteligentniej generować rekordy albo należałoby rozszerzyć/zmodyfikować składnię specyfikacji tak, aby ułatwić kontrolę typów i nazw pól.

Gdyby wymagania zadania nie wskazywały wprost (a tym samym nie groziły utratą punktów) konieczności zdefiniowania `Dir` i `State` w sekcji `domains`, co następnie wymusza ich obsługę również w `parser` i `printer`, pozostałbym przy poprzedniej wersji specyfikacji języka Robot2D. Dla porównania w plikach projektu umieszczę starszą wersję specyfikacji: `robot2d-starszy.txt`. Należy jednak uwzględnić, że pętla główna (plik `Robot2DMain.java`) została zmieniona w nowszej wersji specyfikacji, więc bez dodatkowych modyfikacji bezpośrednie porównanie obu wersji nie będzie całkowicie trywialne.

W ramach przyjętego zakresu funkcjonalnego projekt spełnia wymagania: zapewnia powtarzalny proces budowania, umożliwia uruchamianie interpretera z linii poleceń oraz zawiera zestaw testów potwierdzających poprawność wykonania i spójność semantyk. Naturalnym kierunkiem dalszych prac byłoby rozszerzenie testów o przypadki brzegowe (np. większe wartości parametrów, zagnieżdżone sekwencje) oraz rozwijanie języka o dodatkowe konstrukcje, jeśli wymagania projektu uległyby rozszerzeniu.